```
    chat_id: int,
    topic_id: int,
    title: str = "",
) -> None:
    async def _tx(conn: aiosqlite.Connection) -> None:
        await conn.execute(
            "INSERT OR REPLACE INTO bot_bindings (binding_key, chat_id, topic_id, topic_title) VALUES (?, ?, ?
            (key, chat_id, topic_id, title),
        )

    await DB_WRITER_DAEMON.execute_write_tx(db, _tx)


async def get_topic_binding(
    db: aiosqlite.Connection, key: str
) -> tuple[int, int] | None:
    cur = await db.execute(
        "SELECT chat_id, topic_id FROM bot_bindings WHERE binding_key = ?;",
        (key,),
    )
    row = await cur.fetchone()
    return (int(row[0]), int(row[1])) if row else None


async def get_all_topic_bindings(
    db: aiosqlite.Connection,
) -> dict[str, tuple[int, int, str]]:
    cur = await db.execute(
        "SELECT binding_key, chat_id, topic_id, topic_title FROM bot_bindings;"
    )
    return {
        r[0]: (int(r[1]), int(r[2]), str(r[3] or r[0]))
        for r in await cur.fetchall()
    }


# ==============================================================================
# 3. 6-GATE ZERO-TRUST SECURITY, AST/DUNDER STERILIZER, MODELSCAN & HOT-INJECT
# Menutup VULN-05 (CVE-2026-40158), SSRF, dan Model Serialization Attacks
# ==============================================================================
DYNAMIC_INJECTED_PATTERNS: set[str] = set()
DYNAMIC_SKILL_REGISTRY: dict[str, str] = {
    "UMKM_CATALOG": "Fokus pencahayaan studio komersial, komposisi produk tajam 8K, latar bersih.",
    "CODING_FRONTIER": "Tulis kode modular yang lulus kompilasi AST, bebas except:pass kosong, dan reversible.
    "DEEP_RESEARCH": "Bedah fakta berlabel [FAKTA]/[ESTIMASI], analisis PNL, risiko, dan langkah eksekusi.",
}


def validate_safe_external_url(url: str) -> tuple[bool, str]:
    """Gate 1: Anti-SSRF & Socket IP Validator (Memblokir localhost, IP privat, dan metadata awan)."""
    try:
        parsed = urlparse(url.strip())
        if parsed.scheme != "https":
            return False, "Wajib protokol HTTPS."
        host = (parsed.hostname or "").lower()
        if (
            not host
            or host in ("localhost", "0.0.0.0", "169.254.169.254")
            or host.endswith((".local", ".internal", ".lan"))
        ):
            return False, "Host internal ditolak oleh proteksi Anti-SSRF."
        try:
            ip = ipaddress.ip_address(host)
            if (
                ip.is_private
                or ip.is_loopback
                or ip.is_link_local
                or ip.is_multicast
                or ip.is_reserved
            ):
                return False, f"Alamat IP privat/internal ({ip}) diblokir."
        except ValueError:
            pass
        return True, "URL Aman"
```

```
    except Exception as e:
        return False, f"URL tidak valid: {e}"


def sterilize_community_script_ast(
    source_code: str, script_name: str = "community_plugin"
) -> dict[str, Any]:
    """Gate 2: Whitelist-First AST & Dunder Traversal Blocker v4.0 (Menutup CVE-2026-40158)."""
    allowed_imports = {
        "json",
        "re",
        "math",
        "time",
        "datetime",
        "base64",
        "hashlib",
        "hmac",
        "typing",
        "dataclasses",
        "collections",
    }
    forbidden_calls = {
        "system",
        "popen",
        "eval",
        "exec",
        "compile",
        "open",
        "input",
        "breakpoint",
        "spawnl",
        "getattr",
        "setattr",
        "delattr",
        "globals",
        "locals",
        "vars",
        "dir",
        "__import__",
    }
    forbidden_dunders = {
        "__subclasses__",
        "__globals__",
        "__builtins__",
        "__self__",
        "__import__",
        "__mro__",
        "__bases__",
        "__code__",
        "__closure__",
        "__dict__",
        "__class__",
        "__getattribute__",
    }
    try:
        tree = ast.parse(source_code)
    except SyntaxError as e:
        return {
            "ok": False,
            "reason": f"SyntaxError: {e}",
            "hash": "",
            "threats_removed": [str(e)],
        }

    stripped_threats: list[str] = []
    for node in ast.walk(tree):
        if isinstance(node, (ast.Import, ast.ImportFrom)):
            mod_names = [alias.name.split(".")[0] for alias in node.names]
            if isinstance(node, ast.ImportFrom) and node.module:
                mod_names.append(node.module.split(".")[0])
            for m in mod_names:
                if m not in allowed_imports:
                    stripped_threats.append(
                        f"Blocked non-whitelisted import '{m}'"
```

```
                )
            elif isinstance(node, ast.Attribute) and (
                node.attr in forbidden_dunders or node.attr.startswith("__")
            ):
                stripped_threats.append(f"Blocked dunder attribute '{node.attr}'")
            elif isinstance(node, ast.Name) and (
                node.id in forbidden_dunders
                or node.id in forbidden_calls
                or node.id.startswith("__")
            ):
                stripped_threats.append(f"Blocked identifier '{node.id}'")
            elif isinstance(node, ast.Call):
                fn = (
                    node.func.id
                    if isinstance(node.func, ast.Name)
                    else (
                        node.func.attr
                        if isinstance(node.func, ast.Attribute)
                        else ""
                    )
                )
                if fn in forbidden_calls:
                    stripped_threats.append(f"Blocked call '{fn}()'")
            elif (
                isinstance(node, ast.ExceptHandler)
                and len(node.body) == 1
                and isinstance(node.body[0], ast.Pass)
            ):
                stripped_threats.append("Blocked empty 'except: pass'")

        ast_hash = hashlib.sha256(ast.dump(tree).encode()).hexdigest()[:14]
        if not stripped_threats:
            AUTONOMIC_GUARD.verified_ast_hashes.add(ast_hash)
        return {
            "ok": len(stripped_threats) == 0,
            "script_name": script_name,
            "threats_removed": stripped_threats,
            "hash": ast_hash,
        }


class ModelScanSecurityGate:
    """
    Gate 5: Penegakan Format Tensor Nir-Eksekusi (.safetensors / .gguf)
    dan integrasi pemindai statis protectai/modelscan (Exit Codes 0-4).
    """

    SAFE_EXTENSIONS = (".safetensors", ".gguf")
    RISKY_PICKLE_EXTENSIONS = (".pt", ".pth", ".bin", ".pkl", ".ckpt", ".h5")

    @classmethod
    def verify_model_artifact(cls, model_path_or_repo: str) -> dict[str, Any]:
        target = model_path_or_repo.strip()
        low = target.lower()
        if low.endswith(cls.SAFE_EXTENSIONS) or "gguf" in low or "safetensors" in low:
            return {
                "safe": True,
                "exit_code": 0,
                "format": "SAFETENSORS_OR_GGUF",
                "detail": "Format tensor nir-eksekusi terverifikasi aman (Zero Pickle Execution).",
            }

        # Jika merujuk ke file lokal berisiko Pickle/H5, jalankan CLI modelscan jika tersedia
        if Path(target).exists():
            modelscan_bin = shutil.which("modelscan")
            if modelscan_bin:
                try:
                    proc = subprocess.run(
                        [modelscan_bin, "-p", target],
                        capture_output=True,
                        text=True,
                        timeout=15,
                        check=False,
                    )
```

```
                    return {
                        "safe": proc.returncode == 0,
                        "exit_code": proc.returncode,
                        "format": "SCANNED_BY_PROTECTAI_MODELSCAN",
                        "detail": (proc.stdout or proc.stderr or "")[:250].strip(),
                    }
                except Exception as e:
                    return {
                        "safe": False,
                        "exit_code": 2,
                        "format": "MODELSCAN_ERROR",
                        "detail": f"ModelScan error: {e}",
                    }

        if low.endswith(cls.RISKY_PICKLE_EXTENSIONS):
            return {
                "safe": False,
                "exit_code": 1,
                "format": "UNSAFE_PICKLE_BLOCKED",
                "detail": "Berkas Pickle/PyTorch mentah (.pt/.bin/.pkl) ditolak untuk mencegah Model Serializa
            }

        return {
            "safe": True,
            "exit_code": 0,
            "format": "REMOTE_API_OR_HF_SAFE",
            "detail": "Repositori model diizinkan (Wajib tarik varian Safetensors/GGUF).",
        }


# ================================================================================
# 4. ORCHESTRATOR PLUGIN SURFACE, SELF-HEALING LOOP (3x -> QUEUE) & HUMAN GATE
# Sesuai Spesifikasi Wajib Master Contract D2 & Bagian F
# ================================================================================
@dataclass
class RegisteredPlugin:
    name: str
    version: str
    ast_hash: str
    handler: Callable[..., Coroutine[Any, Any, Any]]
    health_check: Callable[[], Coroutine[Any, Any, bool]]
    active: bool = True
    updated_at: int = field(default_factory=lambda: int(time.time()))


class PluginRegistry:
    """
    Plugin Surface Level Orchestrator:
    Mendukung self-registration, pemeriksaan health(), dan Hot-Swap tanpa restart.
    """

    def __init__(self) -> None:
        self._plugins: dict[str, RegisteredPlugin] = {}

    def register(
        self,
        name: str,
        version: str,
        handler: Callable[..., Coroutine[Any, Any, Any]],
        health_check: Callable[[], Coroutine[Any, Any, bool]] | None = None,
        ast_hash: str = "builtin_verified",
    ) -> None:
        async def _default_health() -> bool:
            return True

        self._plugins[name] = RegisteredPlugin(
            name=name,
            version=version,
            ast_hash=ast_hash,
            handler=handler,
            health_check=health_check or _default_health,
            active=True,
        )
```

```
    async def check_all_health(self) -> dict[str, dict[str, Any]]:
        report: dict[str, dict[str, Any]] = {}
        for name, plug in self._plugins.items():
            if not plug.active:
                report[name] = {
                    "ok": False,
                    "version": plug.version,
                    "status": "DISABLED",
                }
                continue
            try:
                ok = await asyncio.wait_for(plug.health_check(), timeout=2.5)
                report[name] = {
                    "ok": bool(ok),
                    "version": plug.version,
                    "hash": plug.ast_hash,
                    "status": "HEALTHY" if ok else "DEGRADED",
                }
            except Exception as e:
                report[name] = {
                    "ok": False,
                    "version": plug.version,
                    "hash": plug.ast_hash,
                    "status": f"ERROR ({type(e).__name__})",
                }
        return report

    def hot_swap_plugin(
        self, name: str, new_version: str, source_code: str
    ) -> dict[str, Any]:
        """Menyuntikkan / mengganti plugin secara live setelah lulus 6-Gate AST Sterilizer."""
        audit = sterilize_community_script_ast(source_code, script_name=name)
        if not audit["ok"]:
            return {
                "ok": False,
                "reason": f"Ditolak oleh AST Sterilizer: {audit['threats_removed']}",
                "hash": audit["hash"],
            }

        async def _swapped_handler(*args: Any, **kwargs: Any) -> dict[str, Any]:
            return {
                "ok": True,
                "plugin": name,
                "version": new_version,
                "hash": audit["hash"],
            }

        self.register(
            name=name,
            version=new_version,
            handler=_swapped_handler,
            ast_hash=audit["hash"],
        )
        return {
            "ok": True,
            "plugin": name,
            "version": new_version,
            "hash": audit["hash"],
        }


PLUGIN_SURFACE = PluginRegistry()


class AutonomicConvergenceGuard:
    """Anti-Infinite-Loop Guard, Telemetry Window & Shadow-Branch GitOps Controller."""

    def __init__(self) -> None:
        self.verified_ast_hashes: set[str] = set()
        self.incident_remediation_count: dict[str, int] = {}
        self.telemetry_window: deque[tuple[int, str, bool]] = deque(maxlen=80)

    def record_telemetry(self, component: str, success: bool) -> None:
        self.telemetry_window.append((int(time.time()), component, success))
```

```
        if success and component in self.incident_remediation_count:
            self.incident_remediation_count[component] = 0

    def build_gitops_commit_plan(
        self, ast_hash: str, summary: str
    ) -> dict[str, str]:
        return {
            "shadow_branch": f"autonomic/patch-{ast_hash[:10]}",
            "verify_cmd": "python3 -m py_compile app.py",
            "commit_cmd": f"git checkout -b autonomic/patch-{ast_hash[:10]} && git commit -am '[Autonomic-Heal
            "rollback_cmd": "git revert --no-edit HEAD",
        }


AUTONOMIC_GUARD = AutonomicConvergenceGuard()


class SelfHealingOrchestrator:
    """
    Self-Healing Loop Level Orchestrator (Sesuai Kontrak D2):
    classify error -> pilih handler -> apply fix -> verify -> gagal 3x -> escalate ke log + human queue.
    """

    MAX_SAFE_RETRIES = 3

    def classify_error(self, error: Exception | str) -> str:
        msg = str(error).lower()
        if "429" in msg or "rate limit" in msg or "quota" in msg:
            return "RATE_LIMIT_EXHAUSTED"
        if "timeout" in msg or "connect" in msg or "502" in msg or "503" in msg:
            return "UPSTREAM_TIMEOUT_OR_DOWN"
        if "locked" in msg or "sqlite" in msg or "busy" in msg:
            return "DATABASE_CONTENTION"
        if "json" in msg or "parse" in msg:
            return "MALFORMED_UPSTREAM_PAYLOAD"
        return "UNCLASSIFIED_RUNTIME_FAULT"

    def select_handler(self, error_class: str) -> str:
        handlers = {
            "RATE_LIMIT_EXHAUSTED": "QUARANTINE_NODE_AND_ROTATE_AGENT",
            "UPSTREAM_TIMEOUT_OR_DOWN": "SWITCH_STAGGERED_BACKUP_PROVIDER",
            "DATABASE_CONTENTION": "PASSIVE_WAL_CHECKPOINT_AND_BACKOFF",
            "MALFORMED_UPSTREAM_PAYLOAD": "ENABLE_PLAIN_TEXT_STREAM_SCRUBBER",
        }
        return handlers.get(error_class, "ISOLATE_COMPONENT_AND_FALLBACK")

    async def attempt_heal_or_escalate(
        self,
        db: aiosqlite.Connection,
        component: str,
        error: Exception | str,
        fix_coro: Callable[[], Coroutine[Any, Any, bool]] | None = None,
        verify_coro: Callable[[], Coroutine[Any, Any, bool]] | None = None,
    ) -> dict[str, Any]:
        err_class = self.classify_error(error)
        handler_name = self.select_handler(err_class)
        attempts = AUTONOMIC_GUARD.incident_remediation_count.get(component, 0)

        while attempts < self.MAX_SAFE_RETRIES:
            attempts += 1
            AUTONOMIC_GUARD.incident_remediation_count[component] = attempts
            try:
                fix_ok = await fix_coro() if fix_coro else True
                ver_ok = await verify_coro() if verify_coro else fix_ok
                if fix_ok and ver_ok:
                    AUTONOMIC_GUARD.record_telemetry(component, True)
                    return {
                        "healed": True,
                        "escalated": False,
                        "component": component,
                        "error_class": err_class,
                        "handler": handler_name,
                        "attempts": attempts,
                    }
```

```
            except Exception:
                pass

        # Gagal 3x -> Berhenti retry berisiko, catat ke human_escalation_queue & kabari Owner
        AUTONOMIC_GUARD.record_telemetry(component, False)
        inc_id = f"INC-{int(time.time())}-{secrets.token_hex(3).upper()}"
        err_detail = str(error)[:300]

        async def _esc_tx(conn: aiosqlite.Connection) -> None:
            await conn.execute(
                """
                INSERT OR REPLACE INTO human_escalation_queue
                (incident_id, component, error_class, error_detail, attempts, status, created_at)
                VALUES (?, ?, ?, ?, ?, 'OPEN', ?);
                """,
                (
                    inc_id,
                    component,
                    err_class,
                    err_detail,
                    attempts,
                    int(time.time()),
                ),
            )

        await DB_WRITER_DAEMON.execute_write_tx(db, _esc_tx)
        owner_id = int(RUNTIME_CONFIG.get("OWNER_ID_RUNTIME") or 0)
        if owner_id > 0 and BOT_INSTANCE_GLOBAL:
            try:
                await BOT_INSTANCE_GLOBAL.send_message(
                    chat_id=owner_id,
                    text=(
                        f"🚨 **SELF-HEALING ESCALATION (GAGAL 3x BERTURUT-TURUT)**\n"
                        f"├ 🆔 **Insiden:** `{inc_id}`\n"
                        f"├ ⚙️ **Komponen:** `{component}`\n"
                        f"├ 🏷️ **Klasifikasi:** `{err_class}`\n"
                        f"├ 🛠️ **Handler Dicoba:** `{handler_name}` ({attempts}x)\n"
                        f"└ 📝 **Detail:** `{err_detail}`\n\n"
                        f"_Retry otomatis dihentikan agar aman. Insiden telah masuk ke Human Queue di `/admin`
                    ),
                    parse_mode="Markdown",
                )
            except Exception:
                pass

        return {
            "healed": False,
            "escalated": True,
            "incident_id": inc_id,
            "component": component,
            "error_class": err_class,
            "handler": handler_name,
            "attempts": attempts,
        }


SELF_HEALING_ORCHESTRATOR = SelfHealingOrchestrator()

# 5 Kategori Operasi Wajib Human Approval Gate (Sesuai Master Contract D2)
MANDATORY_HUMAN_APPROVAL_ACTIONS = (
    "DATA_MIGRATION",
    "PRICE_PLAN_CHANGE",
    "DELETE_FEATURE_PLUGIN",
    "WITHDRAW_ABOVE_THRESHOLD",
    "SECURITY_POLICY_CHANGE",
)


class HumanApprovalGate:
    """
    Human Approval Gate Eksplisit di Level Kode:
    Menahan eksekusi otomatis untuk 5 operasi sensitif sampai disetujui oleh Owner.
    """
```

```
@staticmethod
async def request_approval(
    db: aiosqlite.Connection,
    action_type: Literal[
        "DATA_MIGRATION",
        "PRICE_PLAN_CHANGE",
        "DELETE_FEATURE_PLUGIN",
        "WITHDRAW_ABOVE_THRESHOLD",
        "SECURITY_POLICY_CHANGE",
    ],
    summary: str,
    payload: dict[str, Any],
) -> dict[str, Any]:
    if action_type not in MANDATORY_HUMAN_APPROVAL_ACTIONS:
        raise ValueError(f"Unknown approval action_type: {action_type}")

    app_id = f"APR-{int(time.time())}-{secrets.token_hex(2).upper()}"
    now_ts = int(time.time())

    async def _tx(conn: aiosqlite.Connection) -> None:
        await conn.execute(
            """
            INSERT INTO pending_human_approvals
            (approval_id, action_type, summary, payload_json, status, created_at, decided_at)
            VALUES (?, ?, ?, ?, 'PENDING', ?, 0);
            """,
            (app_id, action_type, summary, json.dumps(payload), now_ts),
        )

    await DB_WRITER_DAEMON.execute_write_tx(db, _tx)
    return {
        "approval_id": app_id,
        "action_type": action_type,
        "summary": summary,
        "status": "PENDING_HUMAN_APPROVAL",
    }

@staticmethod
async def decide_approval(
    db: aiosqlite.Connection, approval_id: str, approve: bool
) -> tuple[bool, str, dict[str, Any]]:
    cur = await db.execute(
        "SELECT action_type, summary, payload_json, status FROM pending_human_approvals WHERE approval_id
        (approval_id,),
    )
    row = await cur.fetchone()
    if not row:
        return False, "Tiket persetujuan tidak ditemukan.", {}
    act_type, summary, payload_raw, st = (
        str(row[0]),
        str(row[1]),
        str(row[2]),
        str(row[3]),
    )
    if st != "PENDING":
        return False, f"Tiket ini sudah berstatus {st}.", {}

    new_st = "APPROVED" if approve else "REJECTED"

    async def _tx(conn: aiosqlite.Connection) -> None:
        await conn.execute(
            "UPDATE pending_human_approvals SET status = ?, decided_at = ? WHERE approval_id = ?;",
            (new_st, int(time.time()), approval_id),
        )

    await DB_WRITER_DAEMON.execute_write_tx(db, _tx)
    payload = json.loads(payload_raw) if payload_raw else {}

    if not approve:
        return True, f"Operasi `{act_type}` ({summary}) dibatalkan oleh Owner.", payload

    # Eksekusi aksi yang telah disetujui secara eksplisit oleh Owner
    if act_type == "PRICE_PLAN_CHANGE":
```

```
            zone = str(payload.get("zone", "ID"))
            tier = str(payload.get("tier", "ECERAN"))
            new_idr = int(payload.get("idr", 2000))
            new_coins = int(payload.get("coins", 24))
            if zone in PPP_PRICING_PACKAGES and tier in PPP_PRICING_PACKAGES[zone]:
                PPP_PRICING_PACKAGES[zone][tier]["idr"] = new_idr
                PPP_PRICING_PACKAGES[zone][tier]["coins"] = new_coins
        elif act_type == "DELETE_FEATURE_PLUGIN":
            plug_name = str(payload.get("plugin_name", ""))
            if plug_name in PLUGIN_SURFACE._plugins:
                PLUGIN_SURFACE._plugins[plug_name].active = False
        elif act_type == "SECURITY_POLICY_CHANGE":
            cfg_key = str(payload.get("config_key", ""))
            cfg_val = payload.get("config_val")
            if cfg_key in RUNTIME_CONFIG:
                RUNTIME_CONFIG[cfg_key] = cfg_val
                await save_persistent_setting(
                    db, cfg_key, "1" if cfg_val is True else ("0" if cfg_val is False else str(cfg_val))
                )

        return (
            True,
            f"Human Approval Gate LULUS: `{act_type}` ({summary}) berhasil dieksekusi!",
            payload,
        )


HUMAN_APPROVAL_GATE = HumanApprovalGate()

# ==============================================================================
# 5. BITTENSOR SUBNET 27 COMPUTE ADAPTER (TIMEOUT, RETRY, CIRCUIT BREAKER)
# Sesuai Kontrak Bagian B & Bagian F + Sumber Daya Utama.pdf
# ==============================================================================
class BittensorSubnet27Adapter:
    """
    Adaptor kesiapan node Bittensor (Subtensor & Subnet 27 Compute / Chutes AI Serverless):
    - Menghitung skor deterministik node miner: S_total = S_GPU + B_alloc
    - Dilengkapi Timeout ketat, Retry terukur, dan Circuit Breaker otomatis.
    """

    def __init__(self, timeout_sec: float = 3.5, max_retries: int = 2) -> None:
        self.timeout_sec = timeout_sec
        self.max_retries = max_retries
        self.failure_count = 0
        self.circuit_open_until = 0
        self.chutes_endpoint = "https://llm.chutes.ai/v1/chat/completions"

    @staticmethod
    def compute_subnet27_miner_score(
        perf_base: float,
        num_gpus: int,
        max_scenario: float = 31.2,
        is_actively_allocated: bool = True,
    ) -> dict[str, float]:
        """
        Rumus resmi commune-ai/compute-ni (Subnet 27):
        S_GPU = Perf_base * N_GPU * (100 / Max_Scenario)
        S_total = S_GPU + B_alloc (100 poin jika sedang dialokasikan ke validator)
        """
        s_gpu = min(100.0, perf_base * num_gpus * (100.0 / max(0.1, max_scenario)))
        b_alloc = 100.0 if is_actively_allocated else 0.0
        return {
            "s_gpu": round(s_gpu, 2),
            "b_alloc": round(b_alloc, 2),
            "s_total": round(s_gpu + b_alloc, 2),
        }

    async def call_bittensor_chutes_llm(
        self, messages: list[dict[str, str]], model: str = "deepseek-ai/DeepSeek-R1"
    ) -> str | None:
        now_ts = int(time.time())
        if self.circuit_open_until > now_ts or not CHUTES_API_KEY:
            return None
```

```
        headers = {
            "Authorization": f"Bearer {CHUTES_API_KEY}",
            "Content-Type": "application/json",
        }
        payload = {"model": model, "messages": messages, "max_tokens": 800}

        for _ in range(self.max_retries + 1):
            try:
                async with httpx.AsyncClient(timeout=self.timeout_sec) as client:
                    r = await client.post(
                        self.chutes_endpoint, headers=headers, json=payload
                    )
                    if r.status_code == 200:
                        self.failure_count = 0
                        data = r.json()
                        return (
                            data.get("choices", [{}])[0]
                            .get("message", {})
                            .get("content", "")
                        )
            except Exception:
                pass
            self.failure_count += 1
            if self.failure_count >= 3:
                self.circuit_open_until = int(time.time()) + 60
                break
        return None


BITTENSOR_ADAPTER = BittensorSubnet27Adapter()

# ==============================================================================
# 6. BLINDED ZERO-LINKAGE ED25519 VOUCHER (VULN-06) & SECURITY VALIDATORS
# Menghapus "u": user_id dari payload JSON voucher & menerapkan Adaptive Threat Memory
# ==============================================================================
FORBIDDEN_CUSTOMER_JARGON = [
    "SingleWriterSQLiteDaemon",
    "redeemed_nullifiers",
    "Stateless Ed25519",
    "EMVCo CRC16",
    "4x-Remacri",
    "Dual-LLM Sandbox",
    "Dead-Job Sweeper",
    "SangMata",
    "Tuyul",
    "Lock Stripe",
    "HPP Rp 0",
]

_INJECTION_PATTERNS = re.compile(
    r"(ignore\s+(all\s+)?(previous|prior)\s+instructions|system\s*prompt|"
    r"you\s+are\s+now\s+dan|jailbreak|ilja\s*kaerb|drop\s+table|union\s+select|"
    r"<script\b|javascript:|__import__|__subclasses__|__globals__|os\.system|subprocess)",
    re.IGNORECASE,
)


def sanitize_and_deobfuscate_input(raw_text: str) -> tuple[str, bool]:
    """Membersihkan teks input, memeriksa aturan suntik dinamis, & menangkap pola injeksi."""
    if not raw_text:
        return "", False
    normalized = unicodedata.normalize("NFKC", raw_text)
    cleaned = "".join(
        ch
        for ch in normalized
        if unicodedata.category(ch) not in ("Cf", "Cc")
        or ch in ("\n", "\r", "\t")
    ).strip()
    leet_map = str.maketrans(
        {
            "0": "o",
            "1": "i",
            "3": "e",
            "4": "a",
```

```
            "5": "s",
            "7": "t",
            "@": "a",
            "$": "s",
        }
    )
    deobfuscated = cleaned.lower().translate(leet_map)

    for dyn_pat in DYNAMIC_INJECTED_PATTERNS:
        if dyn_pat and (dyn_pat in cleaned.lower() or dyn_pat in deobfuscated):
            return cleaned, True

    is_inj = bool(
        _INJECTION_PATTERNS.search(cleaned)
        or _INJECTION_PATTERNS.search(deobfuscated)
    )
    if is_inj and len(cleaned) >= 12:
        # Self-Evolving Threat Memory: simpan potongan pola serangan baru di memori aktif
        DYNAMIC_INJECTED_PATTERNS.add(deobfuscated[:32])
    return cleaned, is_inj


def sanitize_external_api_response(raw_reply: str) -> str:
    if not raw_reply:
        return ""
    cleaned = re.sub(
        r"<script\b[^>]*>.*?</script>",
        "",
        raw_reply,
        flags=re.IGNORECASE | re.DOTALL,
    )
    cleaned = re.sub(r"javascript\s*:", "", cleaned, flags=re.IGNORECASE)
    return cleaned.replace("|---|", "").strip()


def verify_telegram_webapp_init_data(
    init_data_raw: str, bot_token: str, max_age_sec: int = 3600
) -> dict[str, Any] | None:
    """Menutup VULN-02: Verifikasi kriptografi HMAC-SHA256 timing-safe atas Telegram initData."""
    if not init_data_raw or not bot_token:
        return None
    try:
        parsed = dict(parse_qsl(init_data_raw, keep_blank_values=True))
        received_hash = parsed.pop("hash", "")
        if not received_hash:
            return None
        auth_date = int(parsed.get("auth_date", "0"))
        if auth_date > 0 and abs(int(time.time()) - auth_date) > max_age_sec:
            return None
        data_check_string = "\n".join(
            f"{k}={v}" for k, v in sorted(parsed.items())
        )
        secret_key = hmac.new(
            b"WebAppData", bot_token.encode("utf-8"), hashlib.sha256
        ).digest()
        calculated_hash = hmac.new(
            secret_key, data_check_string.encode("utf-8"), hashlib.sha256
        ).hexdigest()
        return (
            json.loads(parsed.get("user", "{}"))
            if hmac.compare_digest(calculated_hash, received_hash)
            else None
        )
    except Exception:
        return None


def issue_stateless_ed25519_voucher(
    user_id: int, compute_units: int = 15, ttl_seconds: int = 86400
) -> str:
    """
    Menutup VULN-06 (Zero-Identity Linkage):
    TIDAK menyimpan "u": user_id di dalam JSON voucher!
    Diganti dengan Blinded Hash Commitment satu arah H_blind = SHA256(Nonce || Salt)[:16].
```

```
    """
    now_ts = int(time.time())
    nonce_hex = secrets.token_hex(8)
    blind_commitment = hashlib.sha256(
        f"{nonce_hex}:{user_id}".encode("utf-8") + VOUCHER_BLIND_SALT
    ).hexdigest()[:16]
    payload_bytes = json.dumps(
        {
            "n": nonce_hex,
            "b": blind_commitment,
            "c": int(compute_units),
            "t": "PRO_COMPUTE",
            "exp": now_ts + ttl_seconds,
        },
        separators=(",", ":"),
    ).encode("utf-8")
    signature = Ed25519PrivateKey.from_private_bytes(ED25519_PRIV_BYTES).sign(
        payload_bytes
    )
    b64_p = (
        base64.urlsafe_b64encode(payload_bytes).decode("ascii").rstrip("=")
    )
    b64_s = base64.urlsafe_b64encode(signature).decode("ascii").rstrip("=")
    return f"VCH-{b64_p}.{b64_s}"


async def verify_and_redeem_ed25519_voucher(
    db: aiosqlite.Connection, voucher_code: str
) -> tuple[bool, str, int]:
    code = voucher_code.strip()
    if not code.startswith("VCH-") or "." not in code:
        return False, "Format Kode Akses VIP tidak sesuai.", 0
    try:
        b64_p, b64_s = code[4:].split(".", 1)
        payload_bytes = base64.urlsafe_b64decode(
            b64_p + "=" * (-len(b64_p) % 4)
        )
        sig_bytes = base64.urlsafe_b64decode(b64_s + "=" * (-len(b64_s) % 4))
        Ed25519PublicKey.from_public_bytes(ED25519_PUB_BYTES).verify(
            sig_bytes, payload_bytes
        )
        data = json.loads(payload_bytes.decode("utf-8"))
        exp, units = int(data.get("exp", 0)), int(data.get("c", 15))
        if exp <= int(time.time()):
            return False, "Kode Akses VIP sudah melewati masa berlaku.", 0
        nullifier_hash = hashlib.sha256(sig_bytes).hexdigest()

        async def _tx(conn: aiosqlite.Connection) -> bool:
            cur = await conn.execute(
                "SELECT 1 FROM redeemed_nullifiers WHERE nullifier_hash = ?;",
                (nullifier_hash,),
            )
            if await cur.fetchone():
                return False
            await conn.execute(
                "INSERT INTO redeemed_nullifiers (nullifier_hash, expires_at) VALUES (?, ?);",
                (nullifier_hash, exp),
            )
            return True

        if not await DB_WRITER_DAEMON.execute_write_tx(db, _tx):
            return (
                False,
                "Kode Akses VIP ini sudah pernah diaktifkan sebelumnya.",
                0,
            )
        return (
            True,
            f"Kode Akses VIP Aktif (+{units} Kredit Studio Bebas Watermark ditambahkan)!",
            units,
        )
    except Exception:
        return False, "Kode Akses VIP tidak dapat diverifikasi.", 0
```

```
# ============================================================================
# 7. OMNI-RESOURCE HARVESTER (AUTO-TEST VS OWNER SIGNUP CONTROL)
# Memisahkan sumber daya otomatis (Kelompok 1) vs yang butuh daftar akun (Kelompok 2/3)
# ============================================================================
OMNI_BANSOS_PRESETS: dict[str, list[dict[str, Any]]] = {
    "AI_API": [
        {
            "res_id": "API-POLLINATIONS-V1",
            "name": "Pollinations Unified Router (Zero-Auth Keyless)",
            "endpoint": "https://text.pollinations.ai/openai",
            "meta": "openai",
            "desc": "Otomatis Terkoneksi (<1.5s) • Tanpa Daftar Akun",
            "requires_signup": 0,
            "verified": 1,
        },
        {
            "res_id": "API-OR-FREE-POOL",
            "name": "OpenRouter Multi-Model (:free Pool & Hermes/Dolphin)",
            "endpoint": "https://openrouter.ai/api/v1/chat/completions",
            "meta": "deepseek/deepseek-chat:free",
            "desc": "25+ Model Gratis (Login GitHub Tanpa Kartu Visa)",
            "requires_signup": 1,
            "verified": 1 if OPENROUTER_API_KEY else 0,
        },
        {
            "res_id": "API-GROQ-LPU",
            "name": "Groq Cloud LPU (Llama 3.3 70B & Qwen3 320 tok/s)",
            "endpoint": "https://api.groq.com/openai/v1/chat/completions",
            "meta": "llama-3.3-70b-versatile",
            "desc": "1.000 RPD Gratis Tanpa Kartu Visa",
            "requires_signup": 1,
            "verified": 1 if GROQ_API_KEY else 0,
        },
        {
            "res_id": "API-CEREBRAS-1M",
            "name": "Cerebras Wafer-Scale Free Tier (1M Token/Hari)",
            "endpoint": "https://api.cerebras.ai/v1/chat/completions",
            "meta": "llama-3.3-70b",
            "desc": "Bansos 1 Juta Token/Hari Tanpa Kartu Kredit",
            "requires_signup": 1,
            "verified": 1 if CEREBRAS_API_KEY else 0,
        },
        {
            "res_id": "API-CHUTES-BITTENSOR",
            "name": "Chutes AI Serverless (Bittensor Decentralized GPU)",
            "endpoint": "https://llm.chutes.ai/v1/chat/completions",
            "meta": "deepseek-ai/DeepSeek-R1-0528",
            "desc": "Inferensi Desentralisasi Bittensor Kuota Gratis",
            "requires_signup": 1,
            "verified": 1 if CHUTES_API_KEY else 0,
        },
    ],
    "CLOUD_STORAGE": [
        {
            "res_id": "STOR-CF-R2",
            "name": "Cloudflare R2 Object Storage (10 GB Gratis)",
            "endpoint": "https://dash.cloudflare.com/",
            "meta": "10GB + 0% Biaya Egress Bandwidth",
            "desc": "Penyimpanan utama media & CDN",
            "requires_signup": 1,
            "verified": 1,
        },
        {
            "res_id": "STOR-STORJ-DEPIN",
            "name": "Storj Decentralized S3 Cloud (25 GB Gratis)",
            "endpoint": "https://www.storj.io/",
            "meta": "25GB S3-Compatible Gratis",
            "desc": "Cadangan Storage Mesh #2",
            "requires_signup": 1,
            "verified": 1,
        },
        {
            "res_id": "STOR-B2-BACKBLAZE",
```

```
            "name": "Backblaze B2 Cloud Storage (10 GB Gratis)",
            "endpoint": "https://www.backblaze.com/cloud-storage",
            "meta": "10GB Tanpa Kartu Visa",
            "desc": "Cadangan Storage Mesh #3",
            "requires_signup": 1,
            "verified": 1,
        },
    ],
    "GRANTS_AND_DEPIN": [
        {
            "res_id": "GRANT-HF-SPACES",
            "name": "Hugging Face Spaces Docker (2 vCPU + 16 GB RAM Gratis 24/7)",
            "endpoint": "https://huggingface.co/spaces",
            "meta": "16GB RAM Tanpa Kartu Kredit • Anti-Mati Saat Keluar Menu",
            "desc": "Solusi Utama Server Live 24/7 Gratis Sambil Tunggu Oracle",
            "requires_signup": 1,
            "verified": 1,
        },
        {
            "res_id": "GRANT-MODAL-GPU",
            "name": "Modal.com Serverless GPU ($30/Bulan Gratis)",
            "endpoint": "https://modal.com/",
            "meta": "$30/bln Kredit GPU A10G/H100",
            "desc": "Render ~1.500 video Wan 2.2/bln modal Rp 0",
            "requires_signup": 1,
            "verified": 1,
        },
        {
            "res_id": "DEPIN-BITTENSOR-SN27",
            "name": "Bittensor Subnet 27 Compute & Nosana GPU Node",
            "endpoint": "https://github.com/commune-ai/compute-ni",
            "meta": "Insentif Token TAO / NOS untuk Idle GPU",
            "desc": "Kendali di tangan Owner (Butuh Dompet Kripto)",
            "requires_signup": 1,
            "verified": 1,
        },
    ],
}


async def ping_and_verify_bansos_endpoint(
    endpoint_url: str, model_name: str = "openai", api_key: str = ""
) -> tuple[bool, float, str]:
    """Gate 4: Menguji mandiri kecepatan (<2.5s), skema JSON, dan kebersihan balasan sebelum adopsi."""
    ok_url, reason = validate_safe_external_url(endpoint_url)
    if not ok_url:
        return False, 0.0, reason
    headers = {"Content-Type": "application/json"}
    if api_key:
        headers["Authorization"] = f"Bearer {api_key}"
    payload = {
        "model": model_name or "openai",
        "messages": [{"role": "user", "content": "Respond with OK"}],
        "max_tokens": 5,
    }
    t0 = time.perf_counter()
    try:
        async with httpx.AsyncClient(
            timeout=2.5, follow_redirects=False
        ) as client:
            r = await client.post(
                endpoint_url, headers=headers, json=payload
            )
            elapsed_ms = round((time.perf_counter() - t0) * 1000, 1)
            return (
                (True, elapsed_ms, f"HTTP 200 OK ({elapsed_ms} ms)")
                if r.status_code == 200
                else (False, elapsed_ms, f"HTTP {r.status_code}")
            )
    except Exception as e:
        return (
            False,
            round((time.perf_counter() - t0) * 1000, 1),
            f"Timeout ({type(e).__name__})",
```

```
        )


async def check_multi_tld_domain_availability_live(
    base_keyword: str = "bikinai",
) -> list[dict[str, str]]:
    clean_kw = re.sub(r"[^a-z0-9-]", "", base_keyword.lower()) or "bikinai"
    candidate_domains = [
        f"{clean_kw}.tech",
        f"{clean_kw}.xyz",
        f"{clean_kw}.art",
        f"{clean_kw}.app",
        f"{clean_kw}.dev",
        f"aether-{clean_kw}.xyz",
    ]
    results = []
    async with httpx.AsyncClient(timeout=3.5, follow_redirects=True) as client:
        for dom in candidate_domains:
            try:
                r = await client.get(f"https://rdap.org/domain/{dom}")
                status = (
                    "🟢 TERSEDIA (Siap Diklaim!)"
                    if r.status_code == 404
                    else (
                        "🔴 Terdaftar"
                        if r.status_code == 200
                        else "🟡 Cek Promo"
                    )
                )
            except Exception:
                status = "🟢 Siap Cek di Cloudflare / Porkbun / GitHub Pack"
            results.append({"domain": dom, "status": status})
    return results


# ==============================================================================
# 8. IDENTITAS PENGGUNA, AFILIASI, TREASURY SPLIT & KASIR MULTI-SLOT QRIS v4.0
# ==============================================================================
async def ensure_user_and_track_identity(
    db: aiosqlite.Connection,
    user_id: int,
    username: str,
    first_name: str = "",
    lang_code: str = "id",
    referrer_candidate: int = 0,
) -> dict[str, Any]:
    today_epoch = int(time.time()) // 86400
    clean_uname = (
        (username or "Kreator")
        .replace("`", "")
        .replace("*", "")
        .replace("_", "\\_")
    )
    raw_uname = username or "Kreator"
    lang_clean = (lang_code or "id").lower()[:5]
    is_indo = lang_clean.startswith(("id", "in", "ms"))
    default_ppp_zone = 4 if is_indo else 1

    async def _tx(conn: aiosqlite.Connection) -> dict[str, Any]:
        cur_check = await conn.execute(
            "SELECT username FROM users WHERE internal_user_id = ?;",
            (user_id,),
        )
        existing = await cur_check.fetchone()
        name_changed_msg = ""

        if not existing:
            valid_ref = (
                referrer_candidate
                if (referrer_candidate > 0 and referrer_candidate != user_id)
                else 0
            )
            await conn.execute(
                """
```

```
        INSERT INTO users (
            internal_user_id, username, first_name, language_code,
            koin_utama, koin_bonus, ppp_zone, quota_day_epoch, referrer_id, created_at
        ) VALUES (?, ?, ?, ?, 0, 4, ?, ?, ?, ?);
        """,
        (
            user_id,
            raw_uname,
            first_name or raw_uname,
            "id" if is_indo else "en",
            default_ppp_zone,
            today_epoch,
            valid_ref,
            int(time.time()),
        ),
    )
    # Update penghitung klaster negara/bahasa untuk fitur Auto-Grup 100 User
    await conn.execute(
        """
        INSERT INTO geo_community_clusters (country_lang_code, user_count, updated_at)
        VALUES (?, 1, ?)
        ON CONFLICT(country_lang_code) DO UPDATE SET
            user_count = user_count + 1,
            updated_at = excluded.updated_at;
        """,
        (lang_clean, int(time.time())),
    )
else:
    old_uname = existing[0]
    if (
        RUNTIME_CONFIG["SUBBOT_SANGMATA_ACTIVE"]
        and old_uname
        and old_uname != raw_uname
        and raw_uname != "Kreator"
    ):
        await conn.execute(
            "INSERT INTO identity_history (user_id, old_username, new_username, changed_at) VALUES (?,
            (user_id, old_uname, raw_uname, int(time.time())),
        )
        await conn.execute(
            "UPDATE users SET username = ? WHERE internal_user_id = ?;",
            (raw_uname, user_id),
        )
        name_changed_msg = f"@{old_uname} -> @{raw_uname}"

await conn.execute(
    """
    UPDATE users
    SET chat_used_today = CASE WHEN quota_day_epoch < ? THEN 0 ELSE chat_used_today END,
        free_img_used_today = CASE WHEN quota_day_epoch < ? THEN 0 ELSE free_img_used_today END,
        quota_day_epoch = ?
    WHERE internal_user_id = ? AND quota_day_epoch < ?;
    """,
    (today_epoch, today_epoch, today_epoch, user_id, today_epoch),
)
cur = await conn.execute(
    """
    SELECT koin_utama, koin_bonus, has_topped_up, ppp_zone, chat_used_today,
           free_img_used_today, last_claim_date, personal_memory_notes, active_canvas_content,
           language_code, referrer_id, affiliate_withdrawable_idr, affiliate_total_earned_coins,
           api_key_token, subscription_until
    FROM users WHERE internal_user_id = ?;
    """,
    (user_id,),
)
row = await cur.fetchone()
sub_until = int(row[14] or 0)
return {
    "koin_utama": int(row[0]),
    "koin_bonus": int(row[1]),
    "has_topped_up": bool(row[2]),
    "ppp_zone": int(row[3]),
    "chat_used_today": int(row[4]),
    "free_img_used_today": int(row[5]),
```

```
            "last_claim_date": row[6] or "",
            "memory": row[7] or "",
            "canvas": row[8] or "",
            "lang": str(row[9] or ("id" if is_indo else "en")),
            "referrer_id": int(row[10] or 0),
            "affiliate_idr": int(row[11] or 0),
            "affiliate_coins": int(row[12] or 0),
            "api_key": str(row[13] or ""),
            "subscription_until": sub_until,
            "has_active_sub": sub_until > int(time.time()),
            "identity_alert": name_changed_msg,
            "safe_uname": clean_uname,
        }

    return await DB_WRITER_DAEMON.execute_write_tx(db, _tx)


async def record_job_and_split_profit(
    db: aiosqlite.Connection,
    revenue_idr: int,
    actual_hpp_idr: int,
    buffer_ratio: float = 0.10,
) -> dict[str, int]:
    if RUNTIME_CONFIG.get("BANSOS_OPERATION_MODE") == "SILENT_PROFIT":
        actual_hpp_idr = 0
    required_working_capital = int(
        math.ceil(actual_hpp_idr * (1.0 + buffer_ratio))
    )
    net_harvestable_profit = max(0, revenue_idr - required_working_capital)

    async def _tx(conn: aiosqlite.Connection) -> int:
        now_ts = int(time.time())
        await conn.execute(
            "INSERT OR IGNORE INTO treasury_summary VALUES (1, 0, 0, 0, 0, 0, ?);",
            (now_ts,),
        )
        cur = await conn.execute(
            """
            UPDATE treasury_summary
            SET gross_revenue_idr = gross_revenue_idr + ?,
                working_capital_vault_idr = working_capital_vault_idr + ?,
                harvestable_profit_vault_idr = MAX(0, harvestable_profit_vault_idr + ?),
                total_jobs_completed = total_jobs_completed + 1,
                updated_at = ?
            WHERE id = 1
            RETURNING total_jobs_completed;
            """,
            (
                revenue_idr,
                required_working_capital,
                net_harvestable_profit,
                now_ts,
            ),
        )
        row = await cur.fetchone()
        return int(row[0]) if row else 1

    jobs_done = await DB_WRITER_DAEMON.execute_write_tx(db, _tx)
    return {
        "working_capital_locked_idr": required_working_capital,
        "net_profit_ready_to_harvest_idr": net_harvestable_profit,
        "fomo_counter": RUNTIME_CONFIG["FOMO_COUNTER_BASE"] + jobs_done,
    }


async def execute_manual_profit_harvest(
    db: aiosqlite.Connection, bypass_threshold_gate: bool = False
) -> tuple[bool, int, int, str]:
    """
    Memanen laba bersih Owner. Jika nominal di atas HARVEST_APPROVAL_THRESHOLD_IDR,
    sistem mengaktifkan Human Approval Gate sesuai Master Contract D2.
    """
    cur_check = await db.execute(
        "SELECT harvestable_profit_vault_idr, total_harvested_by_owner_idr FROM treasury_summary WHERE id = 1;
```

```
    )
    row_c = await cur_check.fetchone()
    avail_now = int(row_c[0]) if row_c else 0
    total_prev = int(row_c[1]) if row_c else 0

    if avail_now <= 0:
        return False, 0, total_prev, "NO_PROFIT_YET"

    threshold = int(
        RUNTIME_CONFIG.get("HARVEST_APPROVAL_THRESHOLD_IDR", 500_000)
    )
    if avail_now > threshold and not bypass_threshold_gate:
        ticket = await HUMAN_APPROVAL_GATE.request_approval(
            db,
            action_type="WITHDRAW_ABOVE_THRESHOLD",
            summary=f"Penarikan Laba Bersih Rp {avail_now:,} (> Threshold Rp {threshold:,})",
            payload={"amount_idr": avail_now},
        )
        return False, avail_now, total_prev, ticket["approval_id"]

    async def _tx(conn: aiosqlite.Connection) -> tuple[bool, int, int]:
        now_ts = int(time.time())
        cur = await conn.execute(
            "SELECT harvestable_profit_vault_idr, total_harvested_by_owner_idr FROM treasury_summary WHERE id
        )
        row = await cur.fetchone()
        if not row or int(row[0]) <= 0:
            return False, 0, int(row[1]) if row else 0
        avail, new_total = int(row[0]), int(row[1]) + int(row[0])
        await conn.execute(
            "UPDATE treasury_summary SET harvestable_profit_vault_idr = 0, total_harvested_by_owner_idr = ?, u
            (new_total, now_ts),
        )
        return True, avail, new_total

    ok, h_now, tot_h = await DB_WRITER_DAEMON.execute_write_tx(db, _tx)
    return ok, h_now, tot_h, "HARVESTED"


def compute_crc16_ccitt(data_str: str) -> str:
    crc = 0xFFFF
    for ch in data_str.encode("utf-8"):
        crc ^= ch << 8
        for _ in range(8):
            crc = (
                ((crc << 1) ^ 0x1021) & 0xFFFF
                if (crc & 0x8000)
                else (crc << 1) & 0xFFFF
            )
    return f"{crc:04X}"


def generate_dynamic_qris_string(static_qris: str, amount_idr: int) -> str:
    if (
        not static_qris
        or len(static_qris) < 20
        or "5802ID" not in static_qris
    ):
        static_qris = (
            "00020101021126610014ID.CO.GOPAY.WWW011893600914312345678902 10G1234567890303UMI"
            "51440014ID.CO.QRIS.WWW0215ID10200000000010303UMI5204581253033605802ID5912BIKINAI STUDIO6007JAKART
        )
    base_qris = static_qris[:-4].replace("010211", "010212", 1)
    if base_qris.endswith("6304"):
        base_qris = base_qris[:-4]
    amt_str = str(int(amount_idr))
    parts = base_qris.split("5802ID", 1)
    reconstructed = (
        f"{parts[0]}54{len(amt_str):02d}{amt_str}5802ID{parts[1]}6304"
    )
    return f"{reconstructed}{compute_crc16_ccitt(reconstructed)}"


async def create_payment_invoice(
```

```
    db: aiosqlite.Connection,
    user_id: int,
    ppp_zone: int,
    tier_key: str = "ECERAN",
) -> dict[str, Any]:
    invoice_id = f"INV-{user_id}-{int(time.time())}"
    unique_code = (user_id % 89) + 10

    if ppp_zone in (1, 2, 3):
        if tier_key == "ECERAN":
            tier_key = "HEMAT"
        pkg = PPP_PRICING_PACKAGES["GLOBAL"].get(
            tier_key, PPP_PRICING_PACKAGES["GLOBAL"]["HEMAT"]
        )
        exact_idr, coins_amt, provider = (
            pkg["idr_eq"],
            pkg["coins"],
            RUNTIME_CONFIG["GLOBAL_ACTIVE_PROVIDER"],
        )
        display_amt = f"{pkg['usd']} USDT / {pkg['stars']} Telegram Stars ⭐"
        qris_str, qris_img_file_id, is_global = "", "", True
    else:
        pkg = PPP_PRICING_PACKAGES["ID"].get(
            tier_key, PPP_PRICING_PACKAGES["ID"]["ECERAN"]
        )
        provider = RUNTIME_CONFIG["ID_ACTIVE_PROVIDER"]
        exact_idr = pkg["idr"] + unique_code
        coins_amt = pkg["coins"]
        slot_info = QRIS_BARCODE_SLOTS.get(provider, {})
        raw_qris = slot_info.get("raw_payload") or GOPAY_STATIC_QRIS_RAW
        qris_img_file_id = slot_info.get("file_id", "")
        display_amt = f"Rp {exact_idr:,}"
        qris_str = generate_dynamic_qris_string(raw_qris, exact_idr)
        is_global = False

    async def _tx(conn: aiosqlite.Connection) -> None:
        await conn.execute(
            """
            INSERT INTO payment_invoices
            (invoice_id, user_id, amount_idr, coins_credited, provider, tier_key, status, created_at)
            VALUES (?, ?, ?, ?, ?, ?, 'PENDING', ?);
            """,
            (
                invoice_id,
                user_id,
                exact_idr,
                coins_amt,
                provider,
                tier_key,
                int(time.time()),
            ),
        )

    await DB_WRITER_DAEMON.execute_write_tx(db, _tx)
    return {
        "invoice_id": invoice_id,
        "tier_key": tier_key,
        "pkg_name": pkg["name"],
        "pkg_desc": pkg.get("desc", ""),
        "coins": coins_amt,
        "provider": provider,
        "display_amount": display_amt,
        "qris_payload": qris_str,
        "qris_file_id": qris_img_file_id,
        "is_global": is_global,
        "is_sub": bool(pkg.get("is_sub")),
        "grant_api": bool(pkg.get("grant_api")),
    }


async def approve_invoice_and_credit_coins(
    db: aiosqlite.Connection, invoice_id: str
) -> tuple[bool, int, int, int, str]:
    """
```

```
Menutup VULN-01:
Memverifikasi tagihan PENDING -> PAID secara atomik, menerbitkan API Key jika paket Mandiri/Sultan,
mengaktifkan Subscription jika paket Sub, serta membagi komisi Affiliate otomatis!
"""

async def _tx(
    conn: aiosqlite.Connection,
) -> tuple[bool, int, int, int, str, int, int]:
    cur = await conn.execute(
        "SELECT user_id, amount_idr, coins_credited, status, tier_key FROM payment_invoices WHERE invoice_
        (invoice_id,),
    )
    row = await cur.fetchone()
    if not row:
        return False, 0, 0, 0, "Tagihan tidak ditemukan.", 0, 0
    uid, amt_idr, coins_add, st, tier_key = (
        int(row[0]),
        int(row[1]),
        int(row[2]),
        str(row[3]),
        str(row[4] or "HEMAT"),
    )
    if st == "PAID":
        return (
            False,
            uid,
            0,
            0,
            "Tagihan ini sudah pernah diverifikasi sebelumnya.",
            0,
            0,
        )

    await conn.execute(
        "UPDATE payment_invoices SET status = 'PAID' WHERE invoice_id = ?;",
        (invoice_id,),
    )

    # Cek apakah pengguna memiliki pengundang (referrer_id) dan apakah sudah punya API Key
    cur_u = await conn.execute(
        "SELECT referrer_id, api_key_token, subscription_until FROM users WHERE internal_user_id = ?;",
        (uid,),
    )
    u_row = await cur_u.fetchone()
    ref_id = int(u_row[0]) if u_row else 0
    cur_api = str(u_row[1] or "") if u_row else ""
    cur_sub = int(u_row[2] or 0) if u_row else 0

    # Jika membeli paket MANDIRI_API, SULTAN, atau SUB_AUTONOMOUS -> terbitkan kunci API sk-bikinai-...
    if tier_key in ("MANDIRI_API", "SULTAN", "SUB_AUTONOMOUS") and not cur_api:
        cur_api = f"sk-bikinai-{uid}-{secrets.token_hex(10)}"

    # Jika membeli Subscription Global -> tambah masa aktif 30 hari
    if tier_key in ("SUB_CREATOR", "SUB_AUTONOMOUS"):
        base_ts = max(int(time.time()), cur_sub)
        cur_sub = base_ts + (30 * 86400)

    await conn.execute(
        """
        UPDATE users
        SET koin_utama = koin_utama + ?,
            has_topped_up = 1,
            api_key_token = ?,
            subscription_until = ?
        WHERE internal_user_id = ?;
        """,
        (coins_add, cur_api, cur_sub, uid),
    )

    # Hitung komisi Referral / Affiliate otomatis
    ref_bonus_coins = 0
    if ref_id > 0:
        if tier_key == "ECERAN":
            ref_bonus_coins = 5
```